

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

ПРАКТИЧЕСКАЯ РАБОТА 4

По дисциплине Проектирование и реализация баз данных

Тема работы Создание объектов БД

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа К3221

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в инфокоммуникацион-
ных системах

Обучающийся

(дата)

(подпись)

Сакулин И.М.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Войтюк Т.Е.

(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 ЗАДАНИЕ 1	4
2 ЗАДАНИЕ 2	7
3 ЗАДАНИЕ 3	9
4 ЗАДАНИЕ 4	12
5 ЗАДАНИЕ 5	13
6 ЗАДАНИЕ 6	15
7 ЗАДАНИЕ 7	18
ЗАКЛЮЧЕНИЕ	22

ВВЕДЕНИЕ

В отчёте представлено решение практической работы.

Цель – Изучить создание и использование представлений, функций и процедур в PostgreSQL.

Задачи

- Создание представления с помощью графического интерфейса.
- Создание представления на основе представления и изменение базовых таблиц.
- Создание материализованного представления.
- Создание скалярной пользовательской функции.
- Создание собственной скалярной функции.
- Создание хранимой процедуры.
- Создание собственной хранимой процедуры.

1 ЗАДАНИЕ 1

Создание представления с помощью графического интерфейса.

Было открыто окно создания представления (рисунок 1), написано имя (рисунок 2) и SELECT запрос (рисунок 3), изучены остальные вкладки (рисунок 4). На рисунке 5 представлен результат проверки представления.

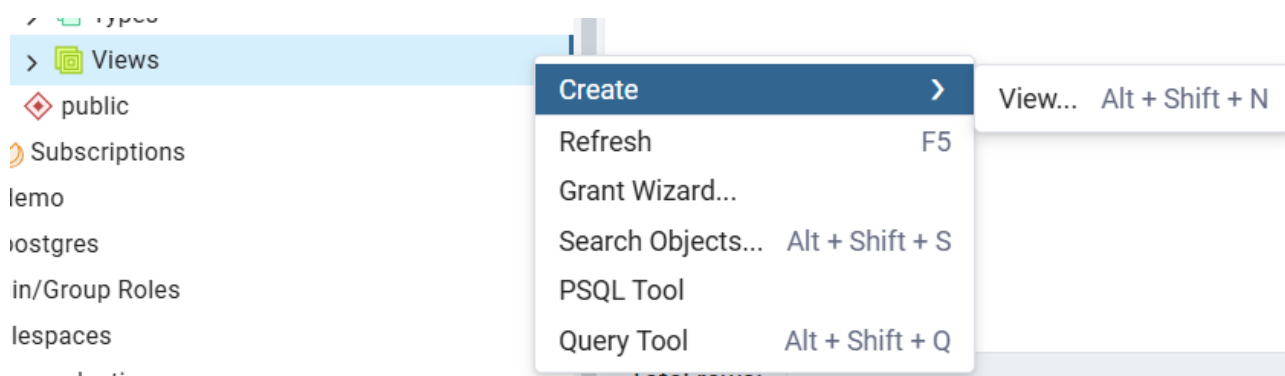


Рисунок 1 — Открытие окна создания представления

Рисунок 2 — Задание имени и комментария к представлению

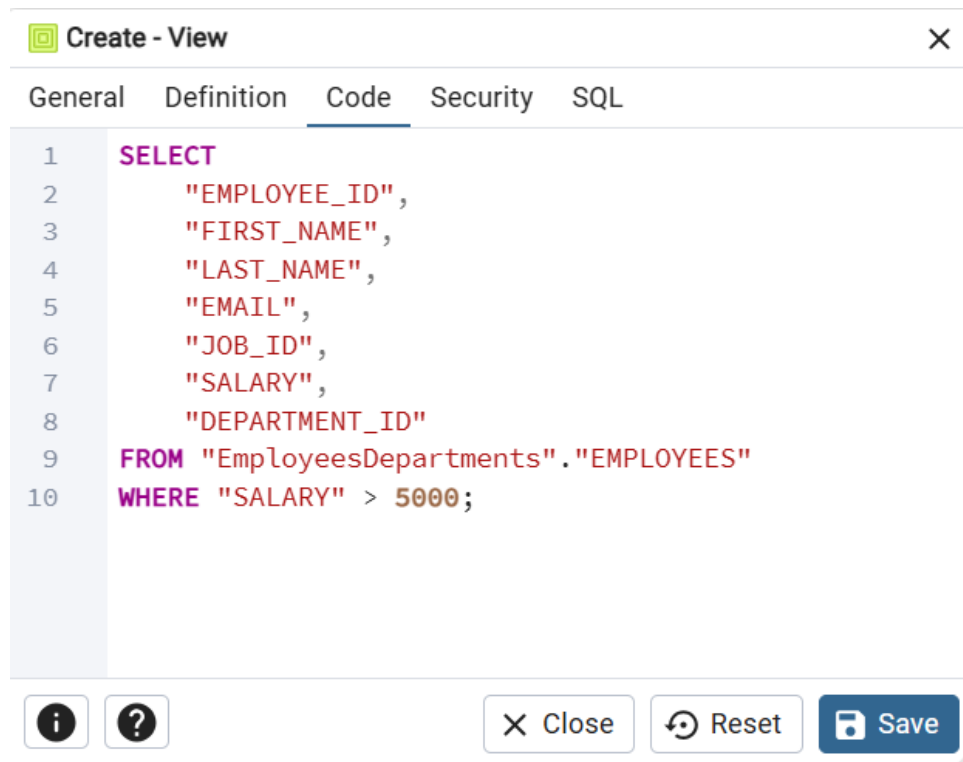


Рисунок 3 — SELECT запрос для представления

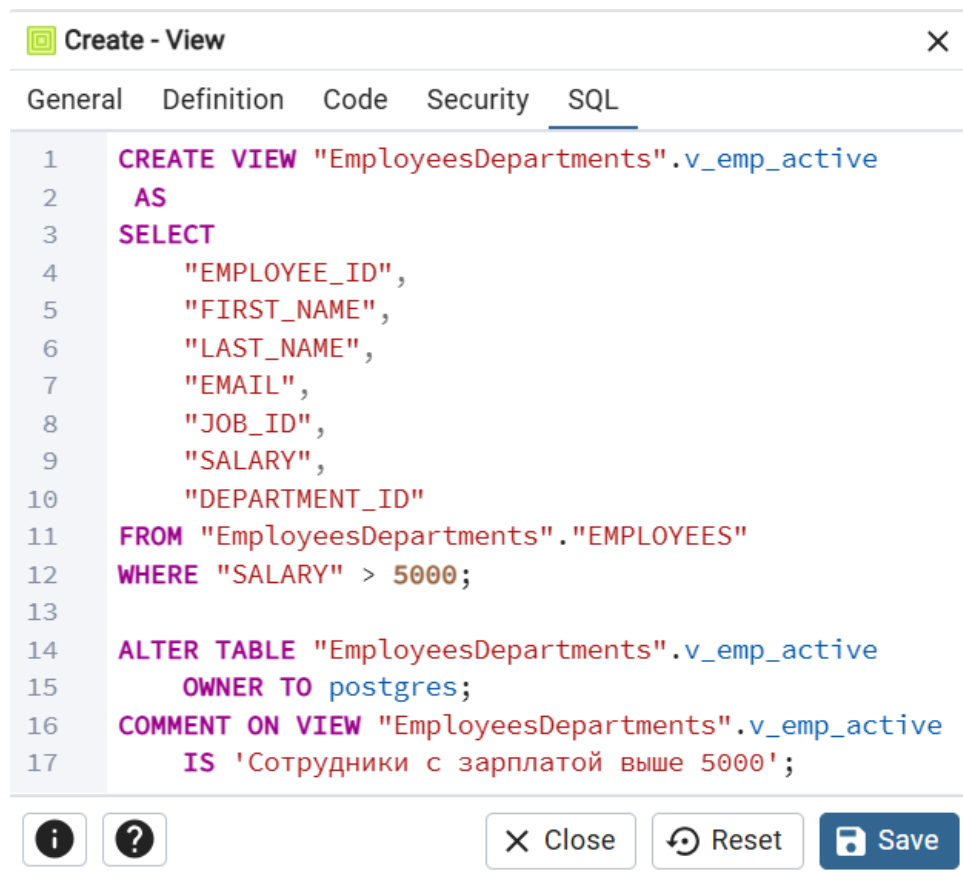


Рисунок 4 — Итоговый SQL запрос

The screenshot shows a PostgreSQL client interface with a sidebar on the left containing a tree view of database objects. The main pane displays the results of a SQL query. The query is: `SELECT * FROM "EmployeesDepartments".v_emp_active`. The results are shown in a table with 8 columns: `EMPLOYEE_ID`, `FIRST_NAME`, `LAST_NAME`, `EMAIL`, `JOB_ID`, `SALARY`, and `DEPARTMENT_ID`. The table contains 14 rows of data.

	EMPLOYEE_ID integer	FIRST_NAME character var	LAST_NAME character varying	EMAIL character varying (255)	JOB_ID character varying	SALARY numeric (8,2)	DEPARTMENT_ID smallint
1	100	Steven	King	SKING	AD_PRES	24000.00	90
2	101	Neena	Kochhar	NKOCHHAR	AD_VP	17000.00	90
3	102	Lex	De Haan	LDEHAAN	AD_VP	17000.00	90
4	205	Shelley	Higgins	SHIGGINS	AC_MGR	12000.00	110
5	206	William	Gietz	WGIETZ	AC_ACCOUNT	8300.00	110
6	149	Eleni	Zlotkey	EZLOTKEY	SA_MAN	10500.00	80
7	174	Ellen	Abel	EABEL	SA_REP	11000.00	80
8	176	Jonathon	Taylor	JTAYLOR	SA_REP	8600.00	80
9	178	Kimberely	Grant	KGRANT	SA_REP	7000.00	[null]
10	124	Kevin	Mourgos	KMOURGOS	ST_MAN	5800.00	50
11	103	Alexander	Hunold	AHUNOLD	IT_PROG	9000.00	60
12	104	Bruce	Ernst	BERNST	IT_PROG	6000.00	60
13	201	Michael	Hartstein	MHARTSTE	MK_MAN	13000.00	20
14	207	Sophia	Barbosa Souza	SBARBOSASOUZA	SR_SA_REP	9500.00	85

Рисунок 5 — Проверка работоспособности

2 ЗАДАНИЕ 2

Создание представления на основе представления и работа с базовыми таблицами.

По запросу с рисунка 6 было создано представление, которое после было протестировано 7.

The screenshot shows the SQL Developer interface. The 'Query' tab is active, displaying the following SQL code:

```
1 CREATE OR REPLACE VIEW "EmployeesDepartments".v_emp_active_info AS
2 SELECT
3     "EMPLOYEE_ID",
4     "FIRST_NAME" || ' ' || "LAST_NAME" AS "FULL_NAME",
5     "EMAIL",
6     "JOB_ID",
7     "SALARY",
8     "DEPARTMENT_ID"
9 FROM "EmployeesDepartments".v_emp_active
10 WHERE "DEPARTMENT_ID" = 80;
```

Below the query editor, the 'Data Output' tab is selected, showing the message: 'CREATE VIEW' and 'Query returned successfully in 64 msec.'

Рисунок 6 — Создания представления на основе представления

The screenshot shows the SQL Developer interface with the 'Query' tab active, displaying the following SQL code:

```
1 SELECT * FROM "EmployeesDepartments".v_emp_active_info;
```

Below the query editor, the 'Data Output' tab is selected, showing a table of results. The table has 7 columns: EMPLOYEE_ID, FULL_NAME, EMAIL, JOB_ID, SALARY, and DEPARTMENT_ID. The results are as follows:

	EMPLOYEE_ID integer	FULL_NAME text	EMAIL character vary	JOB_ID character varyi	SALARY numeric (8,2)	DEPARTMENT_ID smallint
1	149	Eleni Zlotkey	EZLOTKEY	SA_MAN	10500.00	80
2	174	Ellen Abel	EABEL	SA_REP	11000.00	80
3	176	Jonathon Taylor	JTAYLOR	SA_REP	8600.00	80
4	212	Nick Hooper	NHOOPER	SR_SA_REP	9600.00	80

Рисунок 7 — Проверка результата

Для проверки в исходную таблицу и представление-посредник было добавлено две новые записи (рисунок 8). Также были проведены тесты с изменением столбца (рисунок 9).

Query

Query History

1

INSERT INTO "EmployeesDepartments"."EMPLOYEES"

2

("FIRST_NAME", "LAST_NAME", "EMAIL", "JOB_ID", "SALARY", "DEPARTMENT_ID")

3

VALUES ('Ivan', 'Ivanov', 'Iv_Ivn@itmo.ru', 'SA_REP', 6000, 80);

4

5

INSERT INTO "EmployeesDepartments".v_emp_active

6

("FIRST_NAME", "LAST_NAME", "EMAIL", "JOB_ID", "SALARY", "DEPARTMENT_ID")

7

VALUES ('Petr', 'Petrov', 'Pe_Ptr@itmo.ru', 'AD_VP', 22000, 80);

8

9

SELECT * **FROM** "EmployeesDepartments".v_emp_active_info;

Data Output

Messages

Notifications

Рисунок 8 — Добавление записей с проверкой представления

Query	Query History
1	ALTER TABLE "EmployeesDepartments"."EMPLOYEES"
2	ALTER COLUMN "FIRST_NAME" TYPE varchar(100);

Data Output	Messages	Notifications
ERROR: изменить тип столбца, задействованного в представлении или правиле, нельзя правило _RETURN для отношения представление "EmployeesDepartments".v_emp_active зависит от столбца "FIRST_NAME" ОШИБКА: изменить тип столбца, задействованного в представлении или правиле, нельзя SQL state: 0A0000 Detail: правило _RETURN для отношения представление "EmployeesDepartments".v_emp_active зависит от столбца "FIRST_NAME"		

Рисунок 9 — Попытка изменить столбец

3 ЗАДАНИЕ 3

Создание материализованного представления и проверка его обновления. Были удалены представления «v_emp_active» и «v_emp_active_info», так как от него зависит первое, затем было создано материализованное представление «v_emp_active» по такому же SELECT-запросу (рисунок 10). На рисунке 11 представлен результат проверки создания представления.

The screenshot shows the SQL Developer interface. The 'Query' tab is active, displaying the following SQL script:

```
1 DROP VIEW IF EXISTS "EmployeesDepartments".v_emp_active_info;
2 DROP VIEW IF EXISTS "EmployeesDepartments".v_emp_active;
3 CREATE MATERIALIZED VIEW "EmployeesDepartments".v_emp_active AS
4 SELECT
5     "EMPLOYEE_ID",
6     "FIRST_NAME",
7     "LAST_NAME",
8     "EMAIL",
9     "JOB_ID",
10    "SALARY",
11    "DEPARTMENT_ID"
12 FROM "EmployeesDepartments"."EMPLOYEES"
13 WHERE "SALARY" > 5000;
```

Below the query, the 'Data Output' tab shows the result: 'SELECT 26'. A message at the bottom states: 'Query returned successfully in 89 msec.'

Рисунок 10 — Пересоздание представления материальным

The screenshot shows the SQL Developer interface. The 'Query' tab is active, displaying the following SQL script:

```
1 SELECT * FROM "EmployeesDepartments".v_emp_active
2 WHERE "LAST_NAME"='Ivanov';
```

Below the query, the 'Data Output' tab shows the result. A toolbar with icons for various actions is visible above the data table. The data table has the following columns and values:

	EMPLOYEE_ID integer	FIRST_NAME character varyi	LAST_NAME character vary	EMAIL character varying	JOB_ID character	SALARY numeric (	DEPARTMENT_ID smallint
1	250	Ivan	Ivanov	lv_lvn@itmo.ru	SA_REP	6000.00	80

Рисунок 11 — Вывод информации о сотруднике из представления

Данные о сотруднике были обновлены в таблице, они изменились в таблице (рисунок 13), но остались старыми в представлении (рисунок 12).

Query Query History	
1	UPDATE "EmployeesDepartments"."EMPLOYEES"
2	SET "FIRST_NAME"='Petr'
3	WHERE "LAST_NAME"='Ivanov';
4	
5	SELECT * FROM "EmployeesDepartments"."EMPLOYEES"
6	WHERE "LAST_NAME"='Ivanov';

Data Output Messages Notifications	
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL Showing rows: 1 to 1	
	EMPLOYEE_ID [PK] integer
	FIRST_NAME character vary
	LAST_NAME character varyi
	EMAIL character varying
	PHONE_NUMBER character varying (
	HIRE_DATE date
	JOB_ID character v
	SALARY numeric (8
1	250 Petr Ivanov lv_lvn@itmo.ru [null] 2025-11-21 SA_REP 6000.00

Рисунок 12 — Обновление данных в изначальной таблице

Query Query History	
1	SELECT * FROM "EmployeesDepartments".v_emp_active
2	WHERE "LAST_NAME"='Ivanov';

Data Output Messages Notifications	
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL Showing r	
	EMPLOYEE_ID integer
	FIRST_NAME character varyi
	LAST_NAME character varyi
	EMAIL character varying
	JOB_ID character v
	SALARY numeric (8
	DEPARTMENT_ID smallint
1	250 Ivan Ivanov lv_lvn@itmo.ru SA_REP 6000.00 80

Рисунок 13 — Проверка представления до REFRESH

После применения REFRESH данные в представлении также обновлись в соответствии с таблицей (рисунок 14).

Query

Query History

1

REFRESH MATERIALIZED VIEW "EmployeesDepartments".v_emp_active;

2

SELECT * FROM "EmployeesDepartments".v_emp_active

3

WHERE "LAST_NAME"='Ivanov';

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑️

▼

🗄️

▼

⬇️

▼

📈

▼

SQL

Showing 1

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	JOB_ID	SALARY	DEPARTMENT_ID
	integer	character vary	character vary	character varying	character v	numeric (8	smallint
1	250	Petr	Ivanov	Iv_lvn@itmo.ru	SA_REP	6000.00	80

Рисунок 14 — Проверка представления после REFRESH

Материализованное представление храниться в памяти и вызывает SELECT только при REFRESH, а обычное каждый раз при запросе вызывает SELECT.

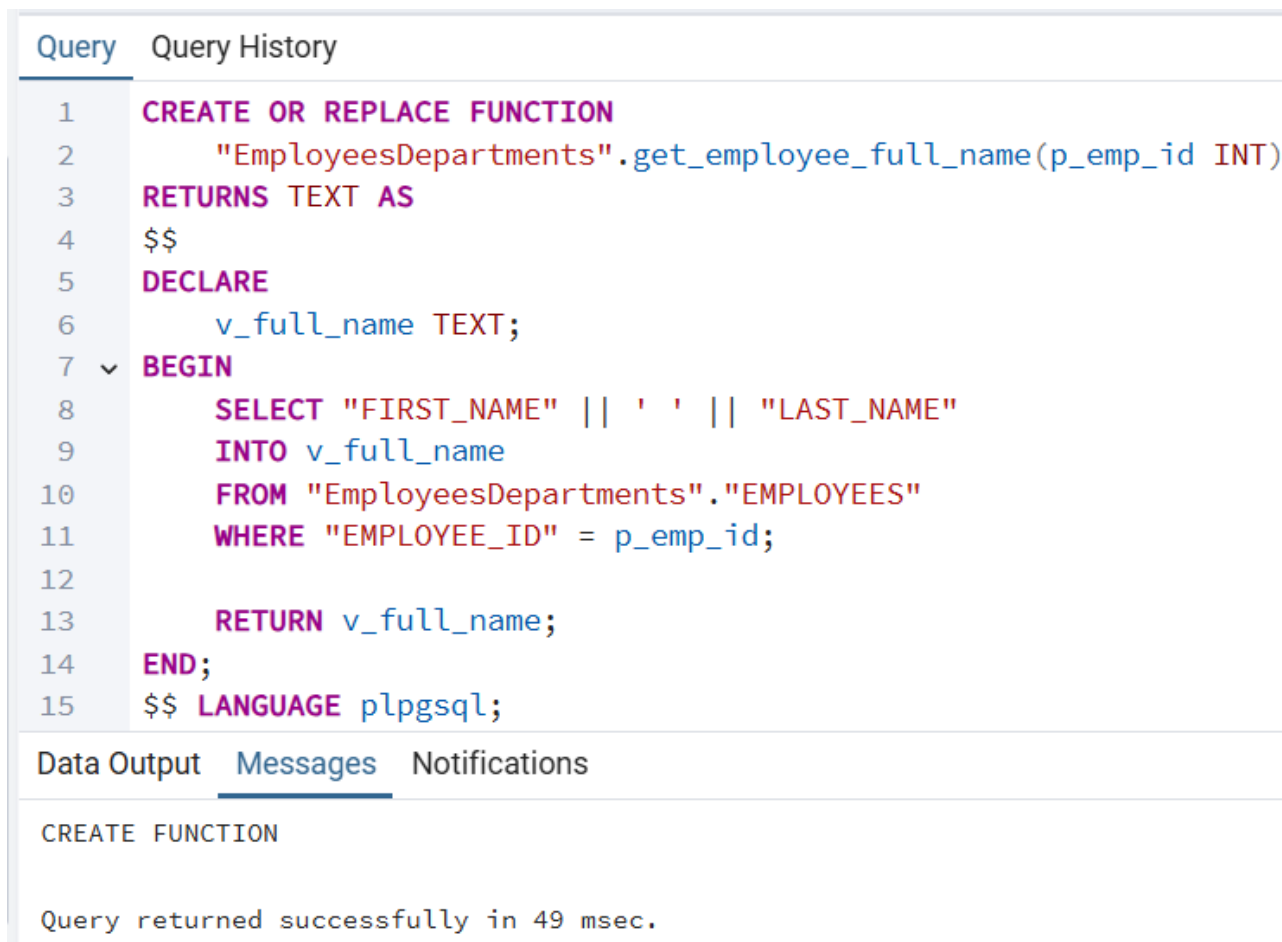
Если нужно скрыть структуру таблиц или защитить от удаления, то идеально подойдёт обычное представление.

Если данные в таблице меняются редко, а запрашиваются часто, то хорошо подойдёт материализованной представление, при применении REFRESH при изменении данных. Особенно хорошо это будет работать при сложных запросах на выборку.

4 ЗАДАНИЕ 4

Создание скалярной пользовательской функции.

По заданию была создана (рисунок 15) и протестирована (рисунок 16) скалярная функция.



The screenshot displays the SQL Developer interface. The 'Query' tab is active, showing a SQL script to create or replace a function. The script defines a function that takes an employee ID and returns their full name by querying the EMPLOYEES table. The 'Messages' tab is also visible, showing a confirmation message that the function was created successfully.

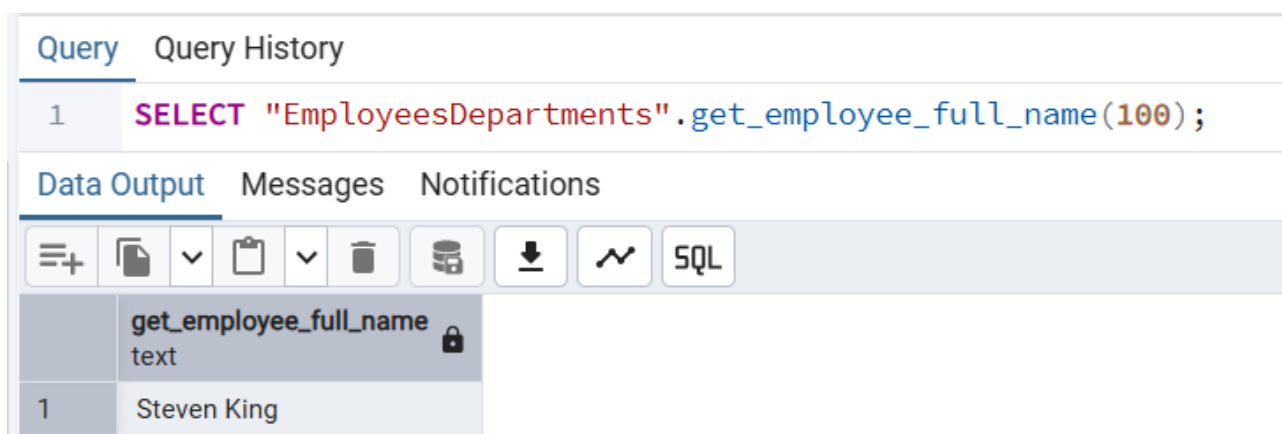
```
1 CREATE OR REPLACE FUNCTION
2     "EmployeesDepartments".get_employee_full_name(p_emp_id INT)
3 RETURNS TEXT AS
4 $$
5 DECLARE
6     v_full_name TEXT;
7 BEGIN
8     SELECT "FIRST_NAME" || ' ' || "LAST_NAME"
9     INTO v_full_name
10    FROM "EmployeesDepartments"."EMPLOYEES"
11   WHERE "EMPLOYEE_ID" = p_emp_id;
12
13     RETURN v_full_name;
14 END;
15 $$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 49 msec.

Рисунок 15 — Создание функции



The screenshot shows the SQL Developer interface with the 'Query' tab active. A SQL query is entered to call the function created in the previous step, passing the value 100. The 'Data Output' tab is selected, showing the result of the function call. The result is a single row with the value 'Steven King'.

```
1 SELECT "EmployeesDepartments".get_employee_full_name(100);
```

Data Output Messages Notifications

get_employee_full_name
text

1	Steven King
---	-------------

Рисунок 16 — Проверка функции

5 ЗАДАНИЕ 5

Создание собственной скалярной функции с параметрами и ветвлением.

Функция сравнения зарплаты сотрудника со средним значением в его отделе представлена на рисунке 17. Первый параметр - ID сотрудника, второй параметр отвечает за вывод в процентах (True) или в единицах (False).

```
2 CREATE OR REPLACE FUNCTION "EmployeesDepartments".get_salary_difference(  
3     p_emp_id INT,  
4     p_percent BOOLEAN  
5 )  
6 RETURNS NUMERIC AS  
7 $$  
8 DECLARE  
9     v_dep_id INT;  
10    v_salary NUMERIC;  
11    v_avg NUMERIC;  
12 BEGIN  
13     SELECT "DEPARTMENT_ID", "SALARY"  
14     INTO v_dep_id, v_salary  
15     FROM "EmployeesDepartments"."EMPLOYEES"  
16     WHERE "EMPLOYEE_ID" = p_emp_id;  
17     IF v_dep_id IS NULL THEN  
18         RETURN 0;  
19     END IF;  
20     SELECT AVG("SALARY") INTO v_avg  
21     FROM "EmployeesDepartments"."EMPLOYEES"  
22     WHERE "DEPARTMENT_ID" = v_dep_id;  
23     IF v_avg IS NULL THEN  
24         RETURN 0;  
25     END IF;  
26     IF p_percent THEN  
27         RETURN ROUND((v_salary - v_avg) / v_avg * 100, 2);  
28     ELSE  
29         RETURN ROUND((v_salary - v_avg), 2);  
30     END IF;  
31 END;  
32 $$ LANGUAGE plpgsql;
```

Рисунок 17 — Создание функции

Проверка при двух режимах работы на рисунках 18 и 19.

Query		Query History
1	SELECT "EmployeesDepartments".get_salary_difference(100, True);	
Data Output		Messages Notifications
≡+ 📄 ▼ 📋 ▼ 🗑 🗄 ⬇ 📈 SQL		
	get_salary_difference	numeric 🔒
1	24.14	

Рисунок 18 — Проверка функции (процентов выше средней)

Query		Query History
1	SELECT "EmployeesDepartments".get_salary_difference(100, False);	
Data Output		Messages Notifications
≡+ 📄 ▼ 📋 ▼ 🗑 🗄 ⬇ 📈 SQL		
	get_salary_difference	numeric 🔒
1	4666.67	

Рисунок 19 — Проверка функции (единиц выше средней)

6 ЗАДАНИЕ 6

Создание хранимой процедуры для изменения заработной платы сотрудника.

По методическим указаниям была создана процедура (рисунок 20).

The screenshot shows a SQL IDE interface with a 'Query' tab selected. The query editor contains the following SQL code:

```
1 CREATE OR REPLACE PROCEDURE "EmployeesDepartments".raise_salary(  
2     p_employee_id INTEGER,  
3     p_percent NUMERIC  
4 ) AS $$  
5 BEGIN  
6     UPDATE "EmployeesDepartments"."EMPLOYEES"  
7     SET "SALARY" = "SALARY" * (1 + p_percent / 100)  
8     WHERE "EMPLOYEE_ID" = p_employee_id;  
9 END;  
10 $$ LANGUAGE plpgsql;
```

Below the query editor, the 'Messages' tab is selected, displaying the following output:

```
CREATE PROCEDURE  
  
Query returned successfully in 45 msec.
```

Рисунок 20 — Создание процедуры

Процедура была проверена с позиционным порядком передачи аргументов (рисунки 21, 22 и 23).

The screenshot shows a SQL IDE interface with a 'Query' tab selected. The query editor contains the following SQL code:

```
1 SELECT "EMPLOYEE_ID", "SALARY"  
2 FROM "EmployeesDepartments"."EMPLOYEES"  
3 WHERE "EMPLOYEE_ID"=100;
```

Below the query editor, the 'Data Output' tab is selected, displaying a table with the following data:

	EMPLOYEE_ID [PK] integer	SALARY numeric (8,2)
1	100	24000.00

Рисунок 21 — Зарплата до вызова процедуры

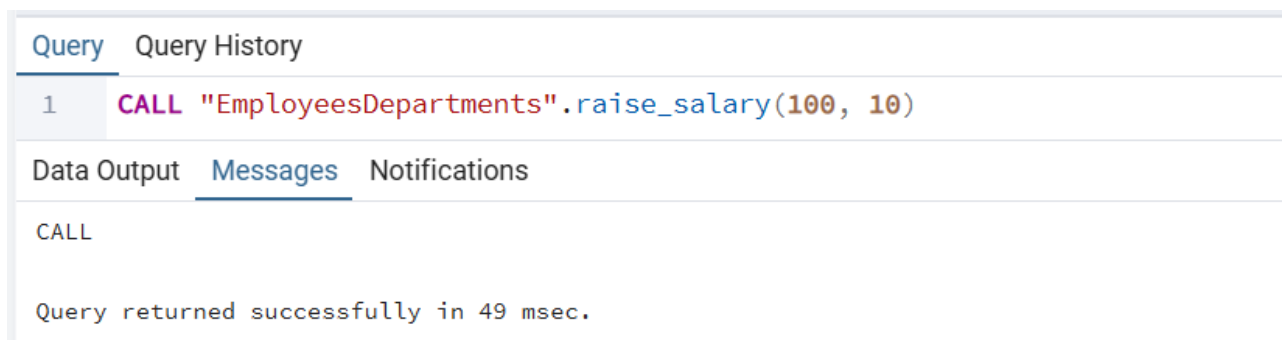


Рисунок 22 — Вызов процедуры

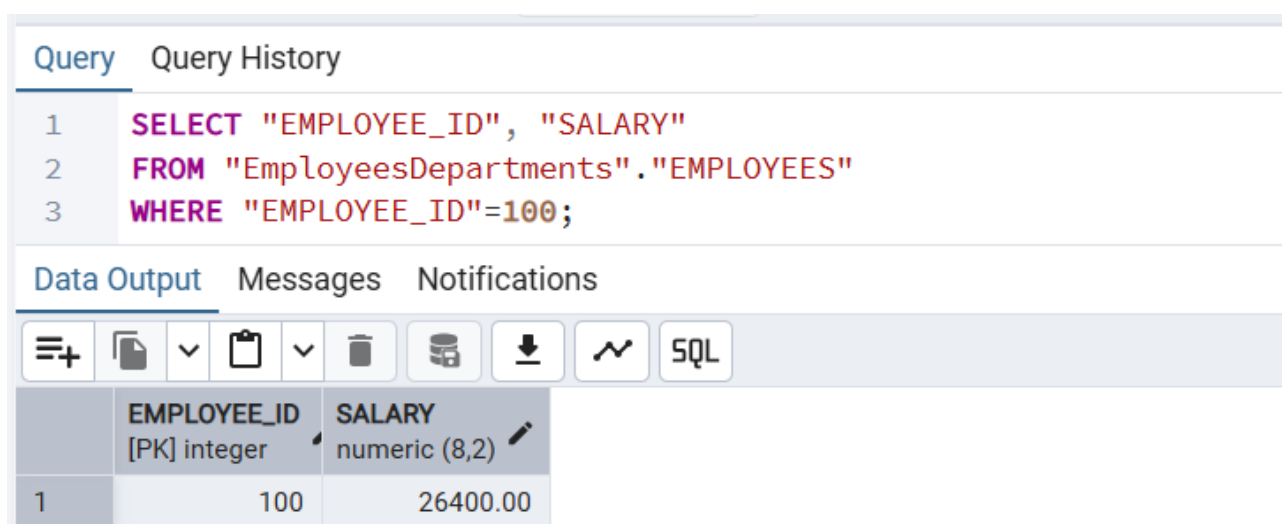


Рисунок 23 — Зарплата после вызова процедуры

Процедура была проверена с передачей аргументов по ключу (рисунки 24, 25 и 26).

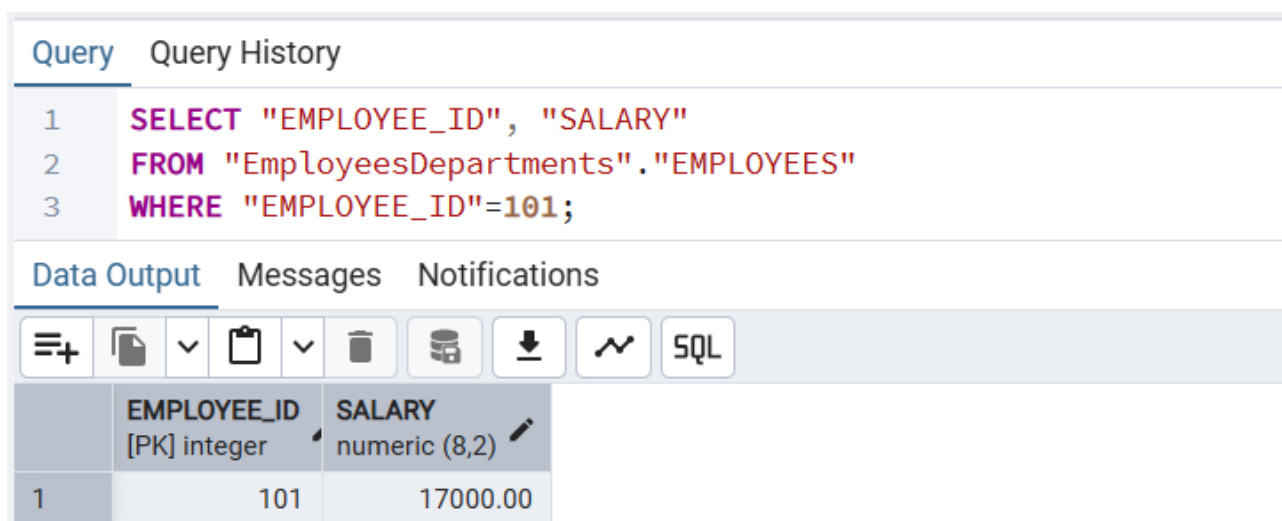


Рисунок 24 — Зарплата до вызова процедуры

Query Query History

```
1 CALL "EmployeesDepartments".raise_salary(p_percent:=10, p_employee_id:=101);
```

Data Output Messages Notifications

CALL

Query returned successfully in 40 msec.

Рисунок 25 — Вызов процедуры

Query Query History

```
1 SELECT "EMPLOYEE_ID", "SALARY"
2 FROM "EmployeesDepartments"."EMPLOYEES"
3 WHERE "EMPLOYEE_ID"=101;
```

Data Output Messages Notifications

EMPLOYEE_ID [PK] integer SALARY numeric (8,2)

1	101	18700.00
---	-----	----------

Рисунок 26 — Зарплата после вызова процедуры

7 ЗАДАНИЕ 7

Создание собственной хранимой процедуры с режимами работы.

Процедура, которая будет увеличивать зарплату все подчинённым руководителю (True) или всем работникам отдела (False), представлена на рисунке 27.

```
1  CREATE OR REPLACE PROCEDURE "EmployeesDepartments".raise_salaries(  
2      p_id INT,  
3      p_is_man BOOLEAN  
4  )  
5  LANGUAGE plpgsql  
6  AS $$  
7  BEGIN  
8      IF p_is_man THEN  
9          -- Увеличить зарплату всем подчинённым указанного менеджера  
10         UPDATE "EmployeesDepartments"."EMPLOYEES"  
11         SET "SALARY" = "SALARY" + 1000  
12         WHERE "MANAGER_ID" = p_id;  
13  
14         ELSE  
15             -- Увеличить зарплату всем работникам отдела  
16             UPDATE "EmployeesDepartments"."EMPLOYEES"  
17             SET "SALARY" = "SALARY" + 2000  
18             WHERE "DEPARTMENT_ID" = p_id;  
19         END IF;  
20     END;  
21     $$;
```

Рисунок 27 — Создание процедуры

Процедура увеличила зарплату всем подчинённым менеджера с ID 100 (рисунки 28, 29 и 30).

Query

Query History

1

2

3

SELECT "EMPLOYEE_ID", "MANAGER_ID", "DEPARTMENT_ID", "SALARY"

FROM "EmployeesDepartments"."EMPLOYEES"

WHERE "MANAGER_ID" = 100;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	EMPLOYEE_ID [PK] integer	MANAGER_ID integer	DEPARTMENT_ID smallint	SALARY numeric (8,2)
1	101	100	90	18700.00
2	102	100	90	17000.00
3	149	100	80	10500.00
4	124	100	50	5800.00
5	201	100	20	13000.00

Рисунок 28 — Зарплаты до вызова процедуры

Query	Query History	
1	CALL "EmployeesDepartments".raise_salaries(100, TRUE);	
Data Output	Messages	Notifications
CALL		
Query returned successfully in 73 msec.		

Рисунок 29 — Вызов процедуры

Query

Query History

1

2

3

SELECT

"EMPLOYEE_ID", "MANAGER_ID", "DEPARTMENT_ID", "SALARY"

FROM

"EmployeesDepartments"."EMPLOYEES"

WHERE

"MANAGER_ID" = 100;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

	EMPLOYEE_ID [PK] integer	MANAGER_ID integer	DEPARTMENT_ID smallint	SALARY numeric (8,2)
1	101	100	90	19700.00
2	102	100	90	18000.00
3	149	100	80	11500.00
4	124	100	50	6800.00
5	201	100	20	14000.00

Рисунок 30 — Зарплаты после вызова процедуры

Процедура увеличила зарплату всем в отделе 110 (рисунки 31, 32 и 33).

Query

Query History

1

SELECT "EMPLOYEE_ID", "MANAGER_ID", "DEPARTMENT_ID", "SALARY"

2

FROM "EmployeesDepartments"."EMPLOYEES"

3

WHERE "DEPARTMENT_ID" = 110;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

	EMPLOYEE_ID [PK] integer	MANAGER_ID integer	DEPARTMENT_ID smallint	SALARY numeric (8,2)
1	205	101	110	12000.00
2	206	205	110	8300.00
3	224	205	110	8100.00
4	231	205	110	5400.00
5	232	205	110	5200.00

Рисунок 31 — Зарплаты до вызова процедуры

Query Query History

```
1 CALL "EmployeesDepartments".raise_salaries(110, FALSE);
```

Data Output Messages Notifications

CALL

Query returned successfully in 41 msec.

Рисунок 32 — Вызов процедуры

Query Query History

```
1 SELECT "EMPLOYEE_ID", "MANAGER_ID", "DEPARTMENT_ID", "SALARY"
2 FROM "EmployeesDepartments"."EMPLOYEES"
3 WHERE "DEPARTMENT_ID" = 110;
```

Data Output Messages Notifications

SQL

	EMPLOYEE_ID [PK] integer	MANAGER_ID integer	DEPARTMENT_ID smallint	SALARY numeric (8,2)
1	205	101	110	14000.00
2	206	205	110	10300.00
3	224	205	110	10100.00
4	231	205	110	7400.00
5	232	205	110	7200.00

Рисунок 33 — Зарплаты после вызова процедуры

ЗАКЛЮЧЕНИЕ

В ходе работы все задания были выполнены успешно. Для каждого задания написаны и протестированы запросы.

При работе возникли следующие трудности: в задании 3.1 не удалось удалить представление с первого раза, потому что от `v_emp_active` зависит `v_emp_active_info`, трудно было придумать оригинальное применение свои процедурам и функциям.

Были рассмотрены новые для меня темы в SQL, которые мне очень понравились, поэтому для себя были придуманы ситуации, где их применить. Рассмотрены также отличия разных видов представлений: материализованное представление храниться в памяти и вызывает SELECT только при REFRESH, а обычное каждый раз при запросе вызывает SELECT, но зато экономит память, защищает некоторые структуры от удаления и скрывает внутреннее устройство таблиц.

В этой лабораторной были впервые опробованы функции и процедуры и условный оператор, рассмотрены их синтаксис и применения.